



TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE SAN JUAN DEL RIO

Materia:

SCC-1012

Inteligencia Artificial

Presenta:

Rangel Olvera Paloma Gpe. B20140953

Maqueda Durazno Alexis 22590043

Mendoza García Daniela 22590044

Hardening Ubuntu

29 / Mayo / 2026



Av. Tecnológico # 2, C.P. 76800, San Juan del Río, Querétaro, México

Tels. 01 (427) 27 24118 Ext. 0

www.tecnm.mx | www.itsanjuan.edu.mx



Contenido

Especificaciones del Entorno del Servidor	2
Protección de acceso al servidor	2
Instalación de Google Authenticator en Linux.....	8
Estructura de almacenamiento para respaldos	16
Instalación de Rclone para integración con Google Drive.....	17
Paso 1. Creación Automática de Carpetas	26
Paso 2. Copia de Archivos de la Aplicación.....	27
Paso 3. Respaldo de la Base de Datos MySQL.....	27
Paso 4. Compresión y Almacenamiento del Respaldo	28
Paso 5. Envío Automático del Respaldo a Google Drive	28
Paso 6. Rotación y Limpieza Automática de Respaldos	29
Paso 7. Verificación Manual de Ejecución del Script	30
Paso 8. Automatización del Respaldo Mediante Cron	33

Especificaciones del Entorno del Servidor

El entorno de producción fue desplegado sobre infraestructura en la nube utilizando la plataforma de DigitalOcean. Para este proyecto se aprovisionó un servidor tipo Droplet, configurado específicamente para alojar la aplicación web y ejecutar las tareas automatizadas de respaldo y administración segura del sistema.

Las características técnicas del servidor implementado se describen a continuación:

- Identificador del Servidor (Hostname): drzn
- Sistema Operativo: Ubuntu 24.04 (LTS) x64
- Recursos de Cómputo (CPU): 1 vCPU (Uso actual estable en ~0.88%)
- Memoria RAM: 1 GB
- Almacenamiento (Volumen): 25 GB Disco SSD.
- Configuración de Red:
 - IPv4 Pública: 67.205.132.100
 - IPv4 Privada (VPC): 10.116.0.2
 - Región del Data Center: NYC1 (Nueva York, EE. UU.).
- Costo de Operación: \$6.00 USD

La configuración seleccionada permite mantener un entorno ligero, estable y adecuado para aplicaciones web con automatización de respaldos y administración remota mediante SSH.

Protección de acceso al servidor

Todo el proceso de configuración, instalación y administración descrito en este manual fue realizado utilizando el usuario Alex, el cual corresponde a una cuenta con privilegios de administrador dentro del servidor.

Configuración y Validación del Firewall UFW

Como parte de las medidas de seguridad implementadas en el servidor Linux, se configuró el firewall UFW, herramienta utilizada para controlar el acceso a los servicios expuestos públicamente dentro del sistema.

Inicialmente, se verificó el estado actual del firewall utilizando el siguiente comando:

sudo ufw status

Tal como se observa en la Figura 1, el firewall se encontraba activo y permitía conexiones únicamente a través del puerto 12222. La validación inicial permitió confirmar que las reglas activas del firewall estaban funcionando correctamente antes de incorporar nuevos servicios o puertos adicionales requeridos por la aplicación.

```
alex@drzn:~$ sudo ufw status
[sudo] password for alex:
Status: active

To                        Action      From
--                        -
12222                     ALLOW      Anywhere
12222/tcp                 ALLOW      Anywhere
12222 (v6)                ALLOW      Anywhere (v6)
12222/tcp (v6)            ALLOW      Anywhere (v6)
```

Figura 1. Verificación inicial del estado del firewall UFW y validación de reglas activas del servidor.

Posteriormente, fue necesario habilitar el puerto 8080/TCP, utilizado por la API de la aplicación web. Para ello, se ejecutó el siguiente comando: **sudo ufw allow 8080/tcp**

Después de agregar la nueva regla, se realizó nuevamente una validación del estado del firewall para confirmar la correcta apertura del puerto. Como se muestra en la Figura 2, el firewall incorporó exitosamente la regla 8080/tcp, manteniendo simultáneamente el acceso SSH seguro mediante el puerto 12222.

Esta configuración permitió restringir el acceso únicamente a los servicios necesarios para la operación de la aplicación, reduciendo la superficie de exposición del servidor ante conexiones no autorizadas.

```
alex@drzn:~$ sudo ufw allow 8080/tcp
Rule added
Rule added (v6)
alex@drzn:~$ sudo ufw status
Status: active

To Action From
--
12222 ALLOW Anywhere
12222/tcp ALLOW Anywhere
8080/tcp ALLOW Anywhere
12222 (v6) ALLOW Anywhere (v6)
12222/tcp (v6) ALLOW Anywhere (v6)
8080/tcp (v6) ALLOW Anywhere (v6)
alex@drzn:~$ |
```

Figura 2. Apertura controlada del puerto 8080/TCP mediante reglas de firewall UFW.

Después de configurar el firewall UFW, se realizaron ajustes adicionales sobre el servicio SSH con el objetivo de fortalecer la seguridad del servidor y reducir riesgos de acceso no autorizado.

Inicialmente, se detectó una duplicación de reglas dentro del firewall relacionadas con el puerto SSH 12222. Como se observa en la Figura 3, se eliminó la regla redundante utilizando el siguiente comando: **sudo ufw delete allow 12222**

Posteriormente, se verificó nuevamente el estado del firewall para confirmar que únicamente permanecieran las reglas necesarias para el acceso remoto y la API de la aplicación.

```
alex@drzn:~$ sudo ufw delete allow 12222
Rule deleted
Rule deleted (v6)
alex@drzn:~$ sudo ufw status
Status: active

To Action From
--
12222/tcp ALLOW Anywhere
8080/tcp ALLOW Anywhere
12222/tcp (v6) ALLOW Anywhere (v6)
8080/tcp (v6) ALLOW Anywhere (v6)
```

Figura 3. Eliminación de reglas duplicadas dentro del firewall UFW.

A continuación, se revisó la configuración activa del servicio SSH mediante el archivo `sshd_config`. Para visualizar únicamente las líneas activas y excluir comentarios, se utilizó el siguiente comando: **sudo cat /etc/ssh/sshd_config | grep -v "^#" | grep -v "^\$"**

Tal como se muestra en la Figura 4, la configuración inicial incluía parámetros relacionados con:

- **Include /etc/ssh/sshd_config.d/*.conf:** Lee configuraciones adicionales de una carpeta separada. Es modular permite agregar reglas sin tocar el archivo principal.
- **Port 12222:** Bien SSH no está en el puerto 22 por defecto. Cambiarlo reduce los ataques automatizados que escanean el puerto 22 masivamente.
- **PermitRootLogin prohibit-password:** root no puede entrar con contraseña, pero sí puede entrar con llave SSH. Lo cambiaremos a no para que root no pueda entrar de ninguna forma.
- **KbdInteractiveAuthentication no:** deshabilita autenticación por teclado interactivo. Lo habilitaremos después solo para el TOTP.
- **UsePAM yes:** permite que PAM maneje la autenticación necesario para el Google Authenticator.
- **X11Forwarding yes:** permite reenviar interfaces gráficas por SSH. Tu servidor no tiene pantalla, esto solo agrega superficie de ataque.
- **PrintMotd no:** no muestra información del sistema al conectarse.
- **AcceptEnv LANG LC_*:** variables de idioma del cliente. No es un riesgo.
- **Subsystem sftp /usr/lib/openssh/sftp-server:** habilita transferencia de archivos por SSH. Lo podemos dejar.

```
alex@drzn:~$ sudo cat /etc/ssh/sshd_config | grep -v "^#" | grep -v "^$"
Include /etc/ssh/sshd_config.d/*.conf
Port 12222
PermitRootLogin prohibit-password
KbdInteractiveAuthentication no
UsePAM yes
X11Forwarding yes
PrintMotd no
AcceptEnv LANG LC_*
Subsystem sftp /usr/lib/openssh/sftp-server
```

Figura 4. Revisión de configuraciones activas del servicio SSH en Linux.

Antes de deshabilitar completamente el acceso por contraseña, se procedió a crear un sistema de autenticación mediante llaves SSH utilizando el algoritmo ed25519.

Como se observa en la Figura 5, la generación de llaves se realizó mediante el siguiente comando: **ssh-keygen -t ed25519 -C "alex@drzn"**

Este procedimiento generó un par de claves pública y privada utilizadas para autenticación segura hacia el servidor Linux sin necesidad de utilizar contraseñas tradicionales.

```
PS C:\Users\adura> ssh-keygen -t ed25519 -C "alex@drzn"
Generating public/private ed25519 key pair.
Enter file in which to save the key (C:\Users\adura\.ssh/id_ed25519):
C:\Users\adura\.ssh/id_ed25519 already exists.
Overwrite (y/n)? n
```

Figura 5. Generación de llaves SSH utilizando el algoritmo ed25519.

Posteriormente, la llave pública generada fue transferida al servidor para habilitar autenticación segura mediante claves SSH (figura 6). Después de completar el proceso de configuración, se validó el acceso remoto sin necesidad de contraseña.

```
PS C:\Users\adura> type C:\Users\adura\.ssh\id_ed25519.pub | ssh -p 12222 alex@67.205.132.100 "mkdir -p ~/.ssh && cat >>
~/.ssh/authorized_keys"
alex@67.205.132.100's password:
```

Figura 6. llave pública generada fue transferida al servidor

Tal como se muestra en la Figura 7, el sistema permitió el acceso exitoso utilizando autenticación basada en llaves, incrementando considerablemente la seguridad del servidor frente a ataques automatizados de fuerza bruta.

```
PS C:\Users\adura> ssh -p 12222 alex@67.205.132.100
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-71-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon May 18 15:52:12 CST 2026

System load:  0.0          Processes:            105
Usage of /:   14.5% of 23.17GB Users logged in:        1
Memory usage: 61%         IPv4 address for eth0: 67.205.132.100
Swap usage:   0%          IPv4 address for eth0: 10.10.0.5

Expanded Security Maintenance for Applications is not enabled.

76 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

4 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

*** System restart required ***
Last login: Mon May 18 15:19:20 2026 from 38.49.139.221
alex@drzn:~$ |
```

Figura 7. Validación de autenticación SSH mediante llaves criptográficas.

Después de validar el acceso seguro mediante llaves SSH, se realizaron modificaciones adicionales sobre el archivo `sshd_config` para endurecer la configuración del servicio. Como se observa en la Figura 8, se modificaron distintos parámetros usando el comando: **sudo nano /etc/ssh/sshd_config**

```
#LoginGraceTime 2m
PermitRootLogin no
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10
#AllowAgentForwarding yes
#AllowTcpForwarding yes
#GatewayPorts no
X11Forwarding no
#X11DisplayOffset 10
#X11UseLocalhost yes
#PermitTTY yes
PrintMotd no

#ClientAliveInterval 0
#ClientAliveCountMax 3
#UseDNS no
#PidFile /run/sshd.pid
#MaxStartups 10:30:100
#PermitTunnel no
#ChrootDirectory none
#VersionAddendum none

# no default banner path
#Banner none

# Allow client to pass locale environment variables
AcceptEnv LANG LC_*

# override default of no subsystems
Subsystem sftp /usr/lib/openssh/sftp-server

# Example of overriding settings on a per-user basis
#Match User anoncvs
#    X11Forwarding no
#    AllowTcpForwarding no
#    PermitTTY no
#    ForceCommand cvs server
MaxAuthTries 3
PasswordAuthentication no
```

Figura 8. Configuración de endurecimiento de seguridad del servicio SSH.

Finalmente, después de guardar los cambios realizados sobre el archivo de configuración, se reinició el servicio SSH para aplicar la nueva configuración del sistema. Tal como se muestra en la Figura 9, se utilizó el siguiente comando: **sudo systemctl restart ssh**

El reinicio permitió activar todas las políticas de seguridad implementadas previamente dentro del servicio SSH del servidor Linux.

```
alex@drzn:~$ sudo nano /etc/ssh/sshd_config
[sudo] password for alex:
alex@drzn:~$ sudo nano /etc/ssh/sshd_config
alex@drzn:~$ sudo systemctl restart ssh
```

Figura 9. Reinicio del servicio SSH para aplicar configuraciones de seguridad avanzadas.

Después de aplicar las configuraciones de seguridad sobre el servicio SSH, se realizó una validación final del archivo sshd_config para comprobar que todos los cambios

hubieran sido aplicados correctamente. Como se observa en la Figura 10, se ejecutó nuevamente el siguiente comando:

```
sudo cat /etc/ssh/sshd_config | grep -v "^#" | grep -v "^$"
```

La verificación permitió confirmar que las políticas de endurecimiento quedaron activas dentro del servidor Linux, incluyendo:

- Deshabilitación del acceso root: PermitRootLogin no
- Desactivación de autenticación por contraseña: PasswordAuthentication no
- Restricción del número máximo de intentos: MaxAuthTries 3
- Desactivación del reenvío gráfico X11: X11Forwarding no

Estas configuraciones fortalecen considerablemente la seguridad del entorno al limitar accesos innecesarios y reducir riesgos asociados a ataques automatizados, fuerza bruta y explotación de servicios no utilizados.

Asimismo, la implementación de autenticación mediante llaves SSH garantiza que únicamente los dispositivos autorizados que posean la clave privada correspondiente puedan establecer conexión con el servidor.



```
alex@drzn:~$ sudo cat /etc/ssh/sshd_config | grep -v "^#" | grep -v "^$"  
Include /etc/ssh/sshd_config.d/*.conf  
Port 12222  
PermitRootLogin no  
KbdInteractiveAuthentication no  
UsePAM yes  
X11Forwarding no  
PrintMotd no  
AcceptEnv LANG LC_*  
Subsystem      sftp      /usr/lib/openssh/sftp-server  
MaxAuthTries 3  
PasswordAuthentication no  
alex@drzn:~$ |
```

Figura 10. Validación final de configuraciones de endurecimiento aplicadas al servicio SSH.

Instalación de Google Authenticator en Linux

Como parte del fortalecimiento de seguridad del servidor, se implementó autenticación multifactor (MFA) utilizando Google Authenticator integrado mediante PAM (*Pluggable*

Authentication Modules). Inicialmente, se instaló el módulo correspondiente utilizando el siguiente comando:

sudo apt install libpam-google-authenticator -y

Tal como se observa en la Figura 11, el sistema descargó e instaló automáticamente las dependencias necesarias para habilitar autenticación TOTP dentro del servicio SSH.

```
alex@drzn:~$ sudo apt install libpam-google-authenticator -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libqrencode4
The following NEW packages will be installed:
  libpam-google-authenticator libqrencode4
0 upgraded, 2 newly installed, 0 to remove and 70 not upgraded.
Need to get 71.8 kB of archives.
After this operation, 212 kB of additional disk space will be used.
Get:1 http://mirrors.digitalocean.com/ubuntu noble/universe amd64 libqrencode4 amd64 4.1.1-1build2 [25.0 kB]
Get:2 http://mirrors.digitalocean.com/ubuntu noble/universe amd64 libpam-google-authenticator amd64 20191231-2build1 [46.8 kB]
Fetched 71.8 kB in 0s (364 kB/s)
Selecting previously unselected package libqrencode4:amd64.
(Reading database ... 111157 files and directories currently installed.)
Preparing to unpack .../libqrencode4_4.1.1-1build2_amd64.deb ...
Unpacking libqrencode4:amd64 (4.1.1-1build2) ...
Selecting previously unselected package libpam-google-authenticator.
Preparing to unpack .../libpam-google-authenticator_20191231-2build1_amd64.deb ...
Unpacking libpam-google-authenticator (20191231-2build1) ...
Setting up libqrencode4:amd64 (4.1.1-1build2) ...
Setting up libpam-google-authenticator (20191231-2build1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Progress: [ 89%] [#####.....]
```

Figura 11. Instalación del módulo PAM Google Authenticator en Ubuntu Server.

Paso 2. Configuración Inicial del Usuario MFA

Después de instalar el módulo PAM, se procedió a configurar Google Authenticator para el usuario del sistema mediante el siguiente comando: **google-authenticator**

Durante la configuración inicial, el sistema solicitó habilitar tokens basados en tiempo, opción necesaria para generar códigos temporales de autenticación de seis dígitos.

Como se muestra en la Figura 12, el sistema generó automáticamente un código QR que posteriormente fue escaneado desde una aplicación de autenticación móvil compatible, como Google Authenticator o Microsoft Authenticator.

Una vez escaneado el código QR, la aplicación comenzó a generar códigos TOTP temporales necesarios para futuras conexiones SSH al servidor.

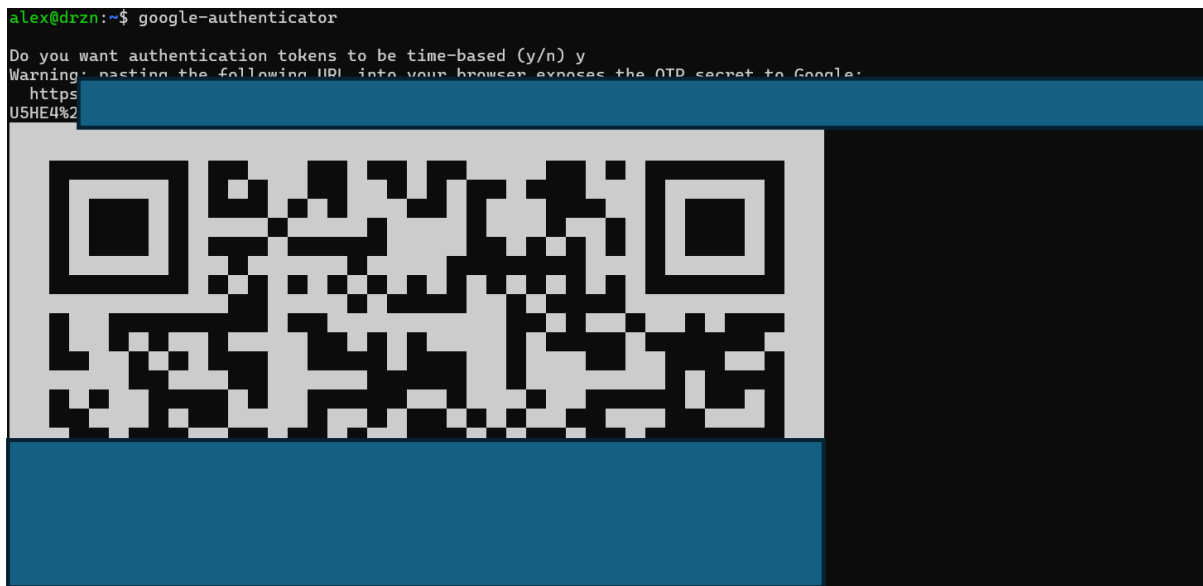


Figura 12. Configuración inicial de autenticación multifactor mediante Google Authenticator.

Posteriormente, el sistema solicitó diversas configuraciones relacionadas con la seguridad de los códigos TOTP generados.

Tal como se observa en la Figura 13, se configuraron las siguientes políticas de seguridad:

- Restricción de reutilización de códigos temporales:

Do you want to disallow multiple uses of the same authentication token?

Respondiste: y

- Ventana estricta de validación de tiempo:

Do you want to increase the window from 3 permitted codes to 17? **Respondiste:** n

- Habilitación de limitación de intentos de autenticación:

Do you want to enable rate-limiting? **Respondiste:** y

Estas configuraciones permiten reducir significativamente riesgos relacionados con ataques de repetición, sincronización de tiempo y fuerza bruta sobre el sistema de autenticación multifactor.

Asimismo, dentro de la misma configuración presentada en la Figura 13, se habilitó el mecanismo de *rate limiting*, restringiendo automáticamente múltiples intentos fallidos consecutivos de autenticación TOTP.

```

Do you want to disallow multiple uses of the same authentication
token? This restricts you to one login about every 30s, but it increases
your chances to notice or even prevent man-in-the-middle attacks (y/n) y

By default, a new token is generated every 30 seconds by the mobile app.
In order to compensate for possible time-skew between the client and the server,
we allow an extra token before and after the current time. This allows for a
time skew of up to 30 seconds between authentication server and client. If you
experience problems with poor time synchronization, you can increase the window
from its default size of 3 permitted codes (one previous code, the current
code, the next code) to 17 permitted codes (the 8 previous codes, the current
code, and the 8 next codes). This will permit for a time skew of up to 4 minutes
between client and server.
Do you want to do so? (y/n) n

If the computer that you are logging into isn't hardened against brute-force
login attempts, you can enable rate-limiting for the authentication module.
By default, this limits attackers to no more than 3 login attempts every 30s.
Do you want to enable rate-limiting? (y/n) y

```

Figura 13. Configuración de políticas de seguridad para autenticación TOTP mediante Google Authenticator.

Paso 3. Integración de Google Authenticator con PAM y SSH

Después de configurar Google Authenticator, se integró el sistema MFA con PAM para que el servicio SSH solicitara automáticamente el código TOTP durante el proceso de autenticación. Inicialmente, se editó el archivo: **sudo nano /etc/pam.d/sshd**

Como se observa en la Figura 14, se agregó la siguiente línea de configuración:

auth required pam_google_authenticator.so

Esta directiva obliga al sistema SSH a validar un código TOTP antes de completar el acceso remoto al servidor.

```

GNU nano 7.2 /etc/pam.d/sshd
# PAM configuration for the Secure Shell service

# Standard Unix authentication.
@include common-auth

# Disallow non-root logins when /etc/nologin exists.
account required pam_nologin.so

# Uncomment and edit /etc/security/access.conf if you need to set complex
# access limits that are hard to express in sshd_config.
# account required pam_access.so

# Standard Unix authorization.
@include common-account

# SELinux needs to be the first session rule. This ensures that any
# lingering context has been cleared. Without this it is possible that a
# module could execute code in the wrong domain.
session [success=ok ignore=ignore module_unknown=ignore default=bad] pam_selinux.so close

# Set the loginuid process attribute.
session required pam_loginuid.so

# Create a new session keyring.
session optional pam_keyinit.so force revoke

# Standard Unix session setup and teardown.
@include common-session

# Print the message of the day upon successful login.
# This includes a dynamically generated part from /run/motd.dynamic
# and a static (admin-editable) part from /etc/motd.
session optional pam_motd.so motd=/run/motd.dynamic
session optional pam_motd.so noudate

# Print the status of the user's mailbox upon successful login.
session optional pam_mail.so standard noenv # [1]

[ Read 55 lines ]
^G Help ^O Write Out ^W Where Is ^R Cut ^T Execute ^C Location ^U Undo ^M Set Mark
^X Exit ^R Read File ^N Replace ^U Paste ^J Justify ^_ Go To Line ^E Redo ^G Copy

```

Figura 14. Integración de Google Authenticator con PAM para autenticación SSH.

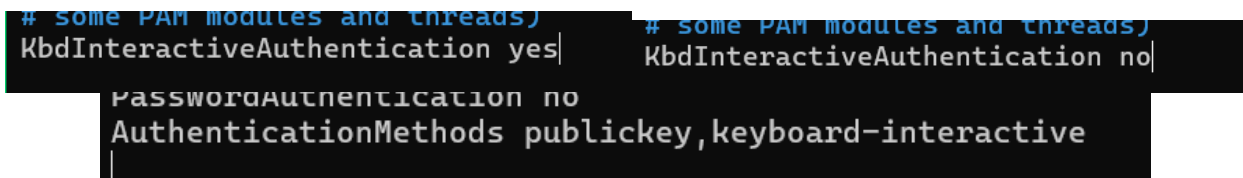
Posteriormente, se realizaron modificaciones adicionales sobre el archivo `sshd_config` para habilitar autenticación interactiva requerida por PAM. Tal como se muestra en la Figura 15, se modificó el parámetro: **KbdInteractiveAuthentication yes**

Asimismo, se agregó la siguiente política de autenticación: **AuthenticationMethods publickey,keyboard-interactive**

Esta configuración obliga al usuario a autenticarse utilizando simultáneamente:

- Llave SSH privada.
- Código TOTP generado desde la aplicación móvil.

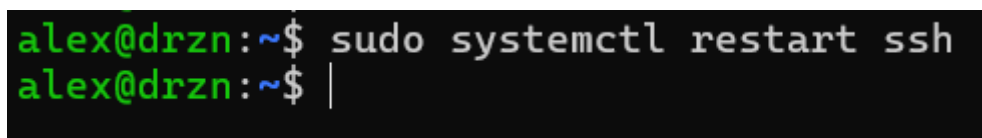
Con ello, el sistema implementa autenticación multifactor real sobre el acceso remoto al servidor Linux.



```
# some PAM modules and threads) KbdInteractiveAuthentication yes| # some PAM modules and threads) KbdInteractiveAuthentication no|
PasswordAuthentication no
AuthenticationMethods publickey,keyboard-interactive
```

Figura 15. Habilitación de autenticación multifactor dentro de la configuración SSH.

Finalmente, después de guardar los cambios realizados en PAM y SSH, se reinició el servicio para aplicar la nueva configuración de seguridad. Como se observa en la Figura 16, se utilizó el siguiente comando: **sudo systemctl restart ssh**



```
alex@drzn:~$ sudo systemctl restart ssh
alex@drzn:~$ |
```

Figura 16. reinició el servicio para aplicar la nueva configuración de seguridad

Posteriormente, se realizó una nueva conexión SSH desde otra terminal para validar el funcionamiento del sistema MFA implementado (Figura 17).

La validación confirmó que el servidor solicitaba correctamente tanto la llave SSH como el código temporal generado desde Google Authenticator antes de permitir el acceso remoto.

```

PS C:\Users\adura> ssh -p 12222 alex@67.205.132.100
(alex@67.205.132.100) Verification code:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-71-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon May 18 16:37:18 CST 2026

System load:  0.0      Processes:    107
Usage of /:   14.5% of 23.17GB   Users logged in:  1
Memory usage: 62%      IPv4 address for eth0: 67.205.132.100
Swap usage:   0%         IPv4 address for eth0: 10.10.0.5

Expanded Security Maintenance for Applications is not enabled.

76 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

4 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

*** System restart required ***
Last login: Mon May 18 16:32:30 2026 from 38.49.139.221
alex@durzi:~$

```

Figura 17. Reinicio y validación final del sistema de autenticación multifactor sobre SSH.

Continuando con la configuración de seguridad del servidor, se implementó autenticación multifactor (TOTP) para el acceso SSH utilizando Google Authenticator mediante PAM (Pluggable Authentication Modules). Con esta configuración, el acceso al servidor requiere tres elementos: la llave privada SSH almacenada en el equipo autorizado, el código dinámico de 6 dígitos generado desde el dispositivo móvil y el uso del puerto SSH configurado para el servicio. De esta manera, aunque un tercero conozca las credenciales del servidor, no podrá acceder sin contar con los demás factores de autenticación.

Como se observa en la Figura 18, la conexión al servidor se realiza utilizando el puerto personalizado 12222 y el código dinámico generado desde la aplicación de autenticación. En la captura también puede apreciarse el acceso exitoso al sistema Ubuntu Server.

```

Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\adura> ssh -p 12222 alex@67.205.132.100
(alex@67.205.132.100) Verification code:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-117-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu May 28 21:36:57 CST 2026

System load:  0.05      Processes:    100
Usage of /:   14.9% of 23.17GB   Users logged in:  0
Memory usage: 57%      IPv4 address for eth0: 67.205.132.100
Swap usage:   0%         IPv4 address for eth0: 10.10.0.5

Expanded Security Maintenance for Applications is not enabled.

64 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

4 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Thu May 28 05:56:07 2026 from 189.203.103.39
alex@durzi:~$

```

Figura 18. Acceso SSH con autenticación TOTP

Posteriormente se creó un usuario adicional llamado dani, destinado a realizar pruebas de acceso mediante autenticación tradicional por contraseña, sin requerir el uso del código TOTP. Para ello se ejecutó el siguiente comando: **sudo adduser dani**

Durante el procedimiento se asignó una contraseña al usuario y se generó automáticamente su directorio personal dentro de /home/dani.

En la Figura 19 se muestra el procedimiento completo de alta del usuario, así como la asignación de contraseña y la creación automática del directorio personal.

```
alex@drzn:~$ sudo adduser dani
[sudo] password for alex:
info: Adding user 'dani' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group 'dani' (1001) ...
info: Adding new user 'dani' (1001) with group 'dani (1001)' ...
info: Creating home directory '/home/dani' ...
info: Copying files from '/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for dani
Enter the new value, or press ENTER for the default
  Full Name []:
  Room Number []:
  Work Phone []:
  Home Phone []:
  Other []:
Is the information correct? [Y/n] y
info: Adding new user 'dani' to supplemental / extra groups 'users' ...
info: Adding user 'dani' to group 'users' ...
alex@drzn:~$
```

Figura 19. Creación del usuario “dani”

Con el objetivo de permitir que el usuario dani accediera únicamente mediante contraseña, se modificó la configuración PAM agregando una excepción para omitir la autenticación TOTP. Asimismo, dentro del archivo sshd_config se definió una política específica para este usuario, permitiendo autenticación por contraseña y limitando el número de intentos de acceso. Finalmente, se reinició el servicio SSH para aplicar los cambios realizados.

La configuración aplicada puede observarse en la Figura 20, donde se muestran las excepciones configuradas tanto en PAM como en sshd_config, además del reinicio del servicio SSH para aplicar correctamente los cambios.

```

auth [success=1 default=ignore] pam_succeed_if.so user ingroup sstotp
auth required pam_google_authenticator.so nullok
auth sufficient pam_permit.so

Match User dani
PasswordAuthentication yes
AuthenticationMethods password
MaxAuthTries 6

alex@drzn:~$ sudo systemctl restart ssh
alex@drzn:~$

```

Figura 20. Configuración PAM y reglas SSH para el usuario “dani”

Después de realizar la configuración, se efectuaron pruebas de conexión desde otra terminal para validar el acceso del usuario dani. La autenticación se realizó correctamente utilizando el puerto SSH configurado y la contraseña asignada al usuario.

Tal como se aprecia en la figura 21, el acceso remoto se realizó exitosamente desde un equipo con Windows utilizando el puerto personalizado del servicio SSH.

```

dani@drzn ~
Microsoft Windows [Versión 10.0.26200.8246]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\xboxo>ssh -p 12222 dani@67.205.132.100
The authenticity of host '67.205.132.100:12222 ([67.205.132.100]:12222)' can't be established.
ED25519 key fingerprint is SHA256:16c500r1UDeHdGhDrZMS1Fe4742Lj+FX8bWu2zD1t7c.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '[67.205.132.100]:12222' (ED25519) to the list of known hosts.
dani@67.205.132.100's password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-117-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue May 26 18:07:45 CST 2026

System load: 0.01          Processes: 106
Usage of /: 14.8% of 23.17GB Users logged in: 1
Memory usage: 55%          IPv4 address for eth0: 67.205.132.100
Swap usage: 0%             IPv4 address for eth0: 10.10.0.5

Expanded Security Maintenance for Applications is not enabled.

60 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

4 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

dani@drzn:~$

```

Figura 21. Validación de acceso del usuario “dani”

Como parte de la configuración de respaldos automáticos desde Windows Server, se creó un usuario dedicado llamado respaldosws, cuyo propósito es almacenar exclusivamente los archivos de respaldo enviados desde el servidor Windows. El usuario fue creado mediante el siguiente comando: **sudo adduser respaldosws**

En la Figura 22 se muestra el procedimiento de creación del usuario y la configuración inicial de su entorno personal.


```

alex@drzn:~$ sudo adduser respaldosws
info: Adding user 'respaldosws' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group 'respaldosws' (1002) ...
info: Adding new user 'respaldosws' (1002) with group 'respaldosws (1002)' ...
info: Creating home directory '/home/respaldosws' ...
info: Copying files from '/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for respaldosws
Enter the new value, or press ENTER for the default
    Full Name []:
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] Y
info: Adding new user 'respaldosws' to supplemental / extra groups 'users' ...
info: Adding user 'respaldosws' to group 'users' ...
alex@drzn:~$

```

Figura 22. Creación del usuario “respaldosws”

Posteriormente se creó la carpeta donde serán almacenados los respaldos provenientes de Windows Server y se asignaron los permisos correspondientes al nuevo usuario. Adicionalmente, se configuró el directorio .ssh para el manejo de llaves públicas y privadas utilizadas durante la autenticación segura.

Finalmente, dentro de sshd_config se configuró una excepción para que el usuario respaldosws únicamente pueda autenticarse mediante llave pública SSH, deshabilitando completamente el acceso por contraseña.

La configuración final puede observarse en la Figura 23, donde se muestra la creación de la carpeta destinada a respaldos, la configuración del directorio .ssh y la restricción de acceso mediante autenticación por llave pública SSH.

```

alex@drzn:~$ # Crear carpeta de respaldos
sudo mkdir -p /home/respaldosws/respaldos-windows
sudo chown respaldosws:respaldosws /home/respaldosws/respaldos-windows
# Crear carpeta .ssh para las llaves
sudo mkdir -p /home/respaldosws/.ssh
sudo chown respaldosws:respaldosws /home/respaldosws/.ssh
sudo chmod 700 /home/respaldosws/.ssh
alex@drzn:~$

```

```

Match User respaldosws
PasswordAuthentication no
AuthenticationMethods publickey|

```

Figura 23. Configuración de carpeta de respaldos y autenticación por llave pública

Estructura de almacenamiento para respaldos

Con el objetivo de mantener organizada la información generada por el sistema, se definió una estructura de directorios dedicada exclusivamente al almacenamiento de respaldos automáticos.

Dentro de la aplicación se configuró una carpeta temporal encargada de generar los archivos iniciales del respaldo. Posteriormente, dichos archivos son comprimidos y almacenados en una ruta externa a la aplicación principal, reduciendo riesgos de pérdida de información ante fallos del sistema o problemas en el entorno web.

La figura 24 muestra la estructura propuesta contempla:

- Respaldo de imágenes utilizadas por la aplicación.
- Exportación de la base de datos MySQL en formato .sql.
- Compresión de archivos mediante formato .tar.gz.
- Separación entre respaldo interno y almacenamiento final.

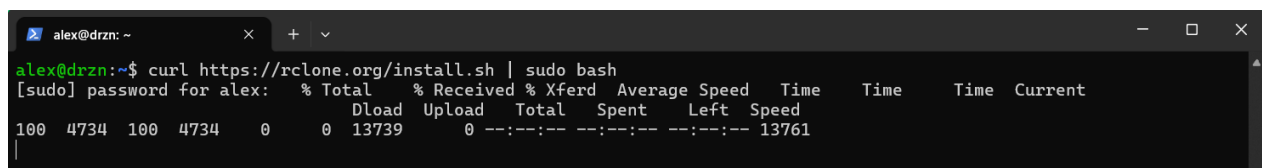
```
/home/alex/backups/           ← fuera de la app (paso 2)
├── 2026-05-17_01-01/
│   ├── fotos.tar.gz          ← imágenes de la app
│   └── database.sql          ← dump de MySQL
/home/alex/unstable/U3---insecure-webapi/backups/ ← paso 1 (dentro de la app)
```

Figura 24. Estructura de carpetas que se crearan

Este enfoque facilita la administración de respaldos diarios y mejora la organización de la información histórica del sistema.

Instalación de Rclone para integración con Google Drive

Como parte de la estrategia de respaldo en la nube, se instaló la herramienta Rclone (Figura 25), utilizada para sincronizar archivos entre el servidor Linux y servicios de almacenamiento externo como Google Drive. La instalación se realizó directamente desde terminal mediante un script oficial ejecutado con privilegios administrativos.



```
alex@drzn: ~  
alex@drzn:~$ curl https://rclone.org/install.sh | sudo bash  
[sudo] password for alex: % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current  
                               Dload  Upload  Total   Spent    Left   Speed  
100  4734  100  4734    0     0  13739      0 --:--:-- --:--:-- --:--:-- 13761
```

Figura 25. Instalación de la herramienta Rclone en Ubuntu Server mediante terminal Linux.

Después de instalar Rclone en el servidor Ubuntu, se inició el asistente interactivo de configuración mediante el comando: **rclone config**

Este procedimiento permitió crear una conexión segura entre el servidor y el servicio de almacenamiento en la nube Google Drive, el cual será utilizado como repositorio externo para los respaldos automáticos generados por la aplicación.

Durante la ejecución inicial del asistente, el sistema detectó que no existía previamente una configuración remota asociada al servidor, por lo que fue necesario crear una nueva conexión (remote). Como se observa en la Figura 26, se definió el identificador gdrive, nombre que será utilizado posteriormente dentro de los procesos automatizados de respaldo para establecer comunicación con Google Drive.

Asimismo, en la misma configuración mostrada en la Figura 26, se seleccionó el tipo de almacenamiento drive, correspondiente al servicio Google Drive. Esta selección permite que el servidor Linux pueda almacenar y sincronizar archivos directamente hacia la nube mediante la herramienta Rclone.

```
alex@drzn:~$ rclone config
2026/05/17 18:53:16 NOTICE: Config file "/home/alex/.config/rclone/rclone.conf" not found - using defaults
No remotes found, make a new one?
n) New remote
s) Set configuration password
q) Quit config
n/s/q> n

Enter name for new remote.
name> gdrive

Option Storage.
Type of storage to configure.
Choose a number from below, or type in your own value.
1 / 1Fichier
```

Figura 26. Creación de una nueva configuración remota en Rclone y selección del servicio Google Drive.

Posteriormente, el asistente solicitó el método de autenticación para vincular el servidor con la cuenta de Google Drive. Tal como se muestra en la Figura 27, se utilizó autenticación web interactiva mediante OAuth2, mecanismo que genera una URL temporal para autorizar el acceso desde un navegador seguro.

Este procedimiento permite establecer una conexión autenticada sin almacenar directamente credenciales sensibles dentro del sistema operativo. Durante el proceso, Rclone quedó en espera del código de autorización generado por Google, completando así el vínculo seguro entre el servidor y el almacenamiento en la nube.

Además, se configuró el alcance (*scope*) con permisos completos sobre los archivos de Google Drive, permitiendo que los procesos automatizados puedan crear, actualizar y transferir respaldos comprimidos generados por la aplicación web.

```
/ Allows read-only access to file metadata but
5 | does not allow any access to read or download file content.
\ (drive.metadata.readonly)
scope> 1

Option service_account_file.
Service Account Credentials JSON file path.
Leave blank normally.
Needed only if you want use SA instead of interactive login.
Leading '~' will be expanded in the file name as will environment variables such as '${RCLONE_CONFIG_DIR}'.
Enter a value. Press Enter to leave empty.
service_account_file>

Edit advanced config?
y) Yes
n) No (default)
y/n> n

Use web browser to automatically authenticate rclone with remote?
* Say Y if the machine running rclone has a web browser you can use
* Say N if running rclone on a (remote) machine without web browser access
If not sure try Y. If Y failed, try N.

y) Yes (default)
n) No
y/n> y

2026/05/17 18:54:19 NOTICE: Make sure your Redirect URL is set to "http://127.0.0.1:53682/" in your custom config.
2026/05/17 18:54:19 ERROR : Failed to open browser automatically (exec: "xdg-open": executable file not found in $PATH)
- please go to the following link: http://127.0.0.1:53682/auth?state=4Dini00CmVUTS6Uim13FFw
2026/05/17 18:54:19 NOTICE: Log in and authorize rclone for access
2026/05/17 18:54:19 NOTICE: Waiting for code...
```

Figura 27. Proceso de autenticación OAuth2 entre Rclone y Google Drive mediante navegador web.

Como parte de la configuración de autenticación de Rclone, se definieron distintos parámetros relacionados con el acceso y permisos hacia Google Drive. En esta etapa, no fue necesario registrar credenciales personalizadas de Google Cloud Platform, debido a que se utilizó la configuración interna proporcionada por Rclone para entornos personales y de baja complejidad administrativa.

Tal como se muestra en la Figura 28, los campos correspondientes a Google Application Client Id y OAuth Client Secret fueron dejados vacíos, permitiendo que la herramienta utilizara las credenciales integradas por defecto. Asimismo, se configuró el alcance (*scope*) con la opción 1, otorgando acceso completo al contenido del almacenamiento en

Google Drive, requisito necesario para la creación y sincronización de respaldos automáticos.

De igual manera, en la misma configuración presentada en la Figura 28, no se utilizó un archivo de credenciales tipo *Service Account*, ya que la autenticación se realizó directamente mediante una cuenta personal de Google. Finalmente, se descartó el uso de configuración avanzada debido a que no eran necesarios parámetros adicionales como límites de transferencia, proxies o configuraciones especiales de red.

```
Storage> drive
Option client_id.
Google Application Client Id
Setting your own is recommended.
See https://rclone.org/drive/#making-your-own-client-id for how to create your own.
If you leave this blank, it will use an internal key which is low performance.
Enter a value. Press Enter to leave empty.
client_id>

Option client_secret.
OAuth Client Secret.
Leave blank normally.
Enter a value. Press Enter to leave empty.
client_secret>

Option scope.
Comma separated list of scopes that rclone should use when requesting access from drive.
Choose a number from below, or type in your own value.
Press Enter to leave empty.
 1 / Full access all files, excluding Application Data Folder.
  \ (drive)
 2 / Read-only access to file metadata and file contents.
```

Figura 28. Configuración de credenciales y permisos de acceso para la conexión entre Rclone y Google Drive.

Posteriormente, Rclone solicitó autorización mediante navegador web para completar el proceso de autenticación OAuth2. Debido a que el servidor Ubuntu no contaba con entorno gráfico, fue necesario crear un túnel SSH desde PowerShell utilizando redirección de puertos locales, permitiendo acceder al enlace de autenticación desde la máquina local del administrador.

Como se observa en la Figura 29, se ejecutó el comando SSH con reenvío de puerto 53682, estableciendo comunicación segura entre el servidor remoto y el navegador local. Esta técnica permitió abrir el enlace generado por Rclone y continuar el proceso de autorización desde un entorno gráfico externo.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\adura> ssh -p 12222 -L 53682:127.0.0.1:53682 alex@67.205.132.100
alex@67.205.132.100's password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-71-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun May 17 18:57:19 CST 2026

System load:  0.0          Processes:      113
Usage of /:   13.9% of 23.17GB   Users logged in:  1
Memory usage: 64%          IPv4 address for eth0: 67.205.132.100
Swap usage:   0%            IPv4 address for eth0: 10.10.0.5

Expanded Security Maintenance for Applications is not enabled.

76 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

# additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

*** System restart required ***
```

Figura 29. Creación de túnel SSH mediante PowerShell para completar la autenticación web de Rclone.

Una vez establecido el túnel SSH, se accedió desde el navegador local al enlace de autenticación generado por Rclone. Tal como se muestra en la Figura 30, el sistema solicitó seleccionar una cuenta de Google para autorizar el acceso al almacenamiento en la nube.

Durante este procedimiento se eligió la cuenta con mayor capacidad disponible de almacenamiento en Google Drive, garantizando suficiente espacio para conservar los respaldos automáticos generados por la aplicación. La autorización concedida permitirá posteriormente realizar transferencias automáticas de archivos comprimidos desde el servidor Linux hacia la nube.

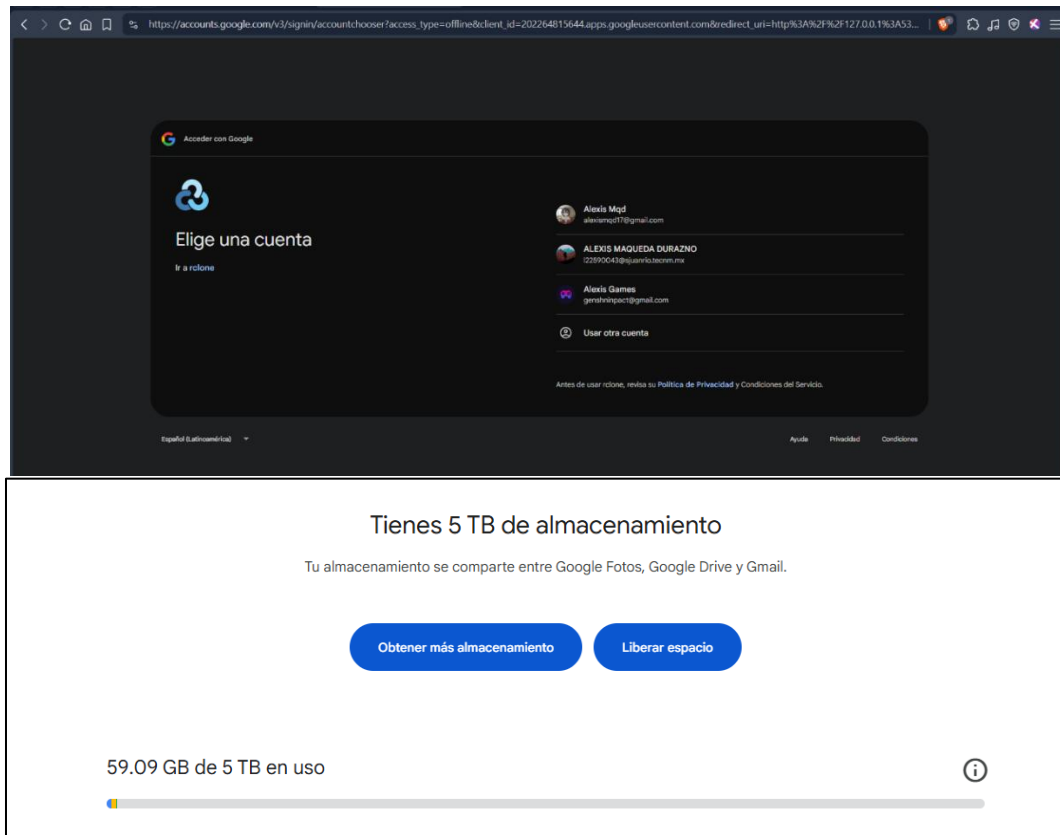


Figura 30. Autorización de cuenta Google y validación del almacenamiento disponible en Google Drive.

Después de seleccionar la cuenta de Google y validar el espacio disponible en Google Drive, el sistema solicitó autorización explícita para que Rclone pudiera acceder al almacenamiento en la nube. Como se observa en la Figura 31, se concedieron permisos de acceso a la cuenta Google, permitiendo que la herramienta pudiera crear, modificar y sincronizar archivos de respaldo desde el servidor Linux.

Esta autorización resulta necesaria para que los procesos automatizados tengan acceso al almacenamiento remoto sin requerir autenticación manual en futuras ejecuciones.

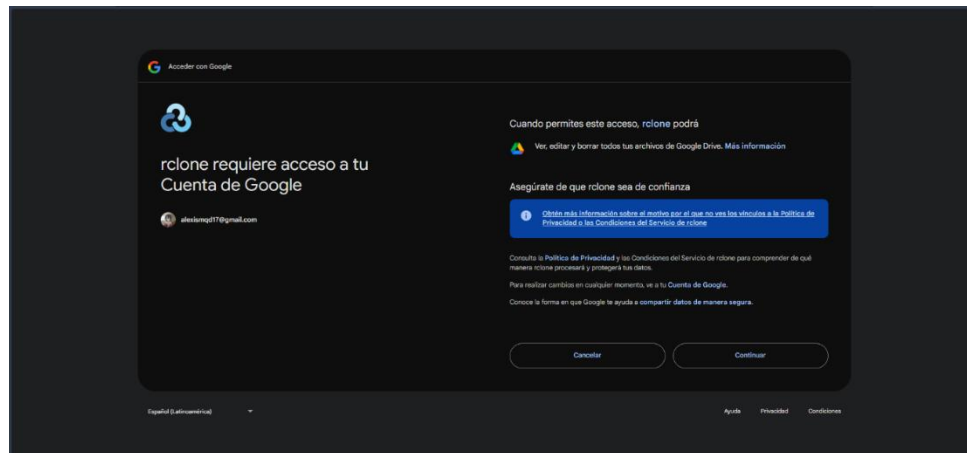


Figura 31. Autorización de permisos de acceso entre Rclone y la cuenta de Google Drive.

Una vez finalizado el proceso de autorización, el navegador mostró el mensaje de confirmación indicando que la autenticación se realizó correctamente. Posteriormente, se regresó a la terminal del servidor para verificar que la configuración remota hubiera sido almacenada correctamente dentro de Rclone.

Tal como se muestra en la Figura 32, se ejecutó el comando: **rclone listremotes**

```
alex@drzn:~$ rclone listremotes
gdrive:
alex@drzn:~$ |
```

Figura 32. Verificación de configuración remota creada correctamente en Rclone

El resultado mostró la conexión gdrive:, confirmando que el almacenamiento remoto quedó configurado satisfactoriamente y disponible para futuras sincronizaciones automáticas(figura 33).

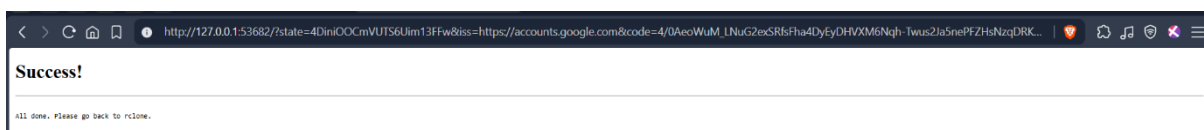


Figura 33. Conexión exitosa

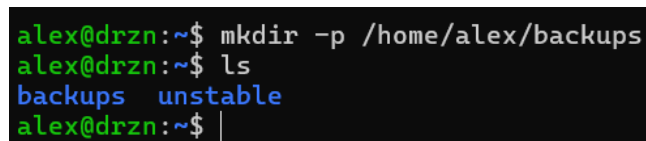
Después de validar la conexión con Google Drive, se identificaron las variables de entorno correspondientes a la conexión de base de datos de la aplicación web. Estas credenciales serían utilizadas posteriormente dentro del script automatizado de respaldo para generar exportaciones de la base de datos MySQL mediante comandos de tipo mysqldump.

Entre los parámetros identificados se encuentran:

- Nombre del host de base de datos.
- Puerto de conexión.
- Nombre de la base de datos.
- Usuario de acceso.
- Contraseña de autenticación.

Posteriormente, se creó el directorio principal destinado al almacenamiento de respaldos locales dentro del servidor Linux. Como se observa en la Figura 34, se utilizó el comando `mkdir -p` para generar la carpeta `/home/alex/backups`, estructura que será utilizada para almacenar temporalmente los archivos comprimidos antes de enviarlos hacia Google Drive.

Asimismo, dentro de la misma sección mostrada en la Figura 34, se inició la creación del archivo `backup.sh`, script encargado de automatizar todo el proceso de respaldo del sistema.



```
alex@drzn:~$ mkdir -p /home/alex/backups
alex@drzn:~$ ls
backups  unstable
alex@drzn:~$ |
```

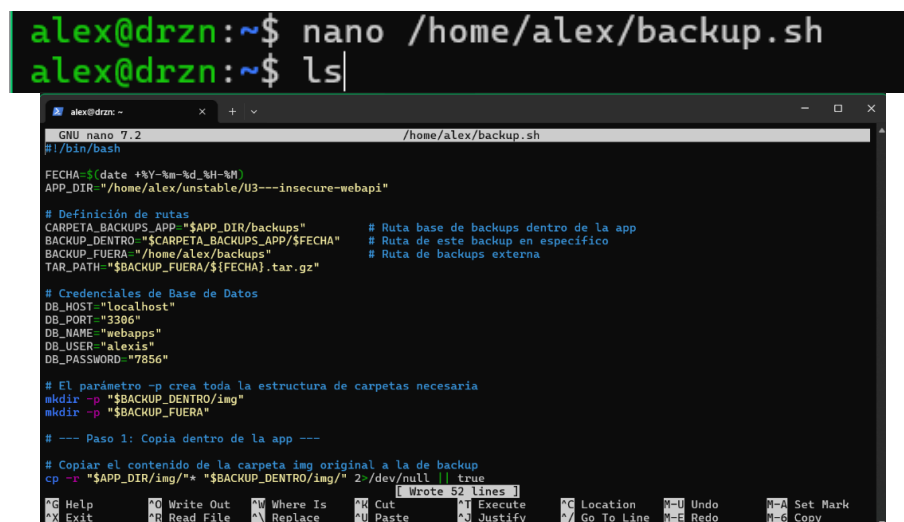
Figura 34. Creación de directorios y archivo de automatización para el sistema de respaldos.

A continuación, se procedió a editar el archivo `backup.sh`, incorporando la lógica principal del sistema de respaldos automatizados. Como se observa en la Figura 35, el script fue desarrollado utilizando Bash sobre entorno Linux, permitiendo automatizar tanto la generación de respaldos locales como la preparación de archivos para sincronización hacia Google Drive.

En la primera sección del script se definió el intérprete de ejecución mediante la directiva `#!/bin/bash`, indicando que todas las instrucciones del archivo deberán ejecutarse utilizando el shell Bash del sistema operativo Ubuntu.

Asimismo, se configuró una variable dinámica denominada `FECHA`, la cual obtiene automáticamente la fecha y hora actual del servidor. Esta variable permite generar nombres únicos para cada respaldo, facilitando la organización cronológica de los archivos comprimidos generados diariamente.

Tal como se muestra en la Figura 35, también se definieron las rutas principales utilizadas por el sistema de respaldo. Entre ellas se encuentra la ubicación de la aplicación web, las carpetas temporales para almacenamiento interno y el directorio externo donde se guardarán finalmente los archivos comprimidos en formato `.tar.gz`.



```
alex@drzn:~$ nano /home/alex/backup.sh
alex@drzn:~$ ls

GNU nano 7.2 /home/alex/backup.sh
#!/bin/bash

FECHA=$(date +%Y-%m-%d_%H-%M)
APP_DIR="/home/alex/unstable/U3---insecure-webapi"

# Definición de rutas
CARPETA_BACKUPS_APP="$APP_DIR/backups" # Ruta base de backups dentro de la app
BACKUP_DENTRO="$CARPETA_BACKUPS_APP/$FECHA" # Ruta de este backup en específico
BACKUP_FUERA="/home/alex/backups" # Ruta de backups externa
TAR_PATH="$BACKUP_FUERA/${FECHA}.tar.gz"

# Credenciales de Base de Datos
DB_HOST="localhost"
DB_PORT="3306"
DB_NAME="webapps"
DB_USER="alexis"
DB_PASSWORD="7856"

# El parámetro -p crea toda la estructura de carpetas necesaria
mkdir -p "$BACKUP_DENTRO/img"
mkdir -p "$BACKUP_FUERA"

# --- Paso 1: Copia dentro de la app ---

# Copiar el contenido de la carpeta img original a la de backup
cp -r "$APP_DIR/img/*" "$BACKUP_DENTRO/img/" 2 /dev/null || true

Wrote 52 lines
[Ctrl] [F] Location [Ctrl] [U] Undo [Ctrl] [A] Set Mark
[Ctrl] [X] Exit [Ctrl] [R] Read File [Ctrl] [W] Where Is [Ctrl] [C] Cut [Ctrl] [V] Paste [Ctrl] [J] Justify [Ctrl] [G] Go To Line [Ctrl] [Z] Redo [Ctrl] [Y] Copy
```

Figura 35. Configuración inicial del script `backup.sh` para automatización de respaldos en Linux.

Dentro de la misma configuración presentada en la Figura 35, se incorporaron las credenciales necesarias para establecer conexión con la base de datos MySQL de la aplicación. Estas variables permiten que posteriormente el script pueda ejecutar procesos de exportación mediante `mysqldump`, generando respaldos completos de la información almacenada en la base de datos.

Posteriormente, en la sección ampliada mostrada en la Figura 36, se observa con mayor detalle la definición de variables correspondientes a:

- Ruta principal de la aplicación.
- Directorios internos de respaldo.
- Carpeta externa de almacenamiento.
- Nombre dinámico del archivo comprimido.
- Parámetros de conexión hacia MySQL.

La utilización de variables centralizadas dentro del script permite mejorar la administración del sistema, simplificando futuras modificaciones de rutas, credenciales o configuraciones generales del proceso de respaldo.

```
#!/bin/bash

FECHA=$(date +%Y-%m-%d_%H-%M)
APP_DIR="/home/alex/unstable/U3---insecure-webapi"

# Definición de rutas
CARPETA_BACKUPS_APP="$APP_DIR/backups"           # Ruta base de backups dentro de la app
BACKUP_DENTRO="$CARPETA_BACKUPS_APP/$FECHA"      # Ruta de este backup en específico
BACKUP_FUERA="/home/alex/backups"                # Ruta de backups externa
TAR_PATH="$BACKUP_FUERA/${FECHA}.tar.gz"

# Credenciales de Base de Datos
DB_HOST="****"
DB_PORT="****"
DB_NAME="webapps"
DB_USER="****"
DB_PASSWORD="****"
```

Figura 36. Definición de variables, rutas de almacenamiento y parámetros de conexión para el sistema automatizado de respaldos.

Paso 1. Creación Automática de Carpetas

Antes de iniciar el respaldo, el script genera automáticamente las carpetas necesarias para almacenar la información temporal y los archivos finales del sistema. Como se observa en la Figura 37, se utilizaron los siguientes comandos:

```
19 # El parámetro -p crea toda la estructura de carpetas necesaria
20 mkdir -p "$BACKUP_DENTRO/img"
21 mkdir -p "$BACKUP_FUERA"
```

Figura 37. Creación de directorios.

El parámetro -p permite crear toda la estructura de carpetas requerida sin generar errores en caso de que los directorios ya existan previamente.

Esta configuración asegura que el proceso automatizado pueda ejecutarse múltiples veces sin interrupciones relacionadas con la estructura de almacenamiento.

Paso 2. Copia de Archivos de la Aplicación

Después de crear las carpetas, el sistema realiza la copia de los archivos principales de la aplicación web. Tal como se muestra en la Figura 38, se utilizó el siguiente comando:

```
25 # Copiar el contenido de la carpeta img original a la de backup
26 cp -r "$APP_DIR/img/"* "$BACKUP_DENTRO/img/" 2>/dev/null || true
27
```

Figura 38. Copia automática de archivos de la aplicación hacia el directorio temporal de respaldo mediante comando cp -r

El comando cp -r realiza una copia recursiva de todos los archivos y subdirectorios contenidos dentro de la carpeta img.

Asimismo, la instrucción 2>/dev/null || true evita que errores menores interrumpan la ejecución del script, permitiendo que el proceso continúe de forma estable aun cuando algún archivo no pueda copiarse temporalmente.

Paso 3. Respaldo de la Base de Datos MySQL

Posteriormente, el script genera una exportación completa de la base de datos utilizando la herramienta mysqldump. Como se observa al final de la Figura 39, se ejecutó el siguiente comando:

```
28 # Extraer y guardar la base de datos MySQL
29 mysqldump -h $DB_HOST -P $DB_PORT -u $DB_USER -p$DB_PASSWORD $DB_NAME > "$BACKUP_DENTRO/database.sql"
30
31
```

Figura 39. Generación automática del respaldo de la base de datos MySQL mediante mysqldump.

Este procedimiento crea un archivo database.sql que contiene toda la estructura y registros de la base de datos de la aplicación.

La generación del respaldo SQL permite restaurar la información del sistema en caso de pérdida de datos, errores críticos o migraciones futuras.

Paso 4. Compresión y Almacenamiento del Respaldo

Una vez recopilada toda la información de la aplicación, el sistema genera un archivo comprimido en formato .tar.gz. Como se muestra en la Figura 40, se utilizó el siguiente comando:

```
34 # Comprimimos la carpeta con el nombre de la fecha, ubicándonos primero en la carpeta de backups
35 tar -czf "$TAR_PATH" -C "$CARPETA_BACKUPS_APP" "$FECHA"
36
```

Figura 40. Compresión automática de respaldos en formato .tar.gz mediante la herramienta tar.

La herramienta tar permite empaquetar múltiples archivos y directorios en un único archivo comprimido. La utilización de compresión Gzip reduce el tamaño final del respaldo y facilita tanto el almacenamiento como la transferencia hacia servicios externos en la nube.

Asimismo, el parámetro -C permite posicionarse temporalmente dentro de la carpeta objetivo antes de realizar la compresión, evitando almacenar rutas completas innecesarias dentro del archivo final.

Paso 5. Envío Automático del Respaldo a Google Drive

Después de generar el archivo comprimido, el sistema ejecuta automáticamente la sincronización hacia Google Drive mediante Rclone. Tal como se observa en la Figura 41, se utilizó el siguiente comando:

```
40 # Sube el archivo .tar.gz (Drive acumulará todos los archivos sin borrar los viejos)
41 rclone copy "$TAR_PATH" gdrive:backups/
42
```

Figura 41. Sincronización automática de archivos comprimidos hacia Google Drive mediante rclone copy.

Este procedimiento copia automáticamente el archivo .tar.gz generado hacia la carpeta backups/ configurada previamente dentro de Google Drive.

La sincronización automática permite mantener copias externas de seguridad fuera del servidor principal, incrementando la disponibilidad y protección de la información del sistema.

Paso 6. Rotación y Limpieza Automática de Respaldos

Con el objetivo de evitar el crecimiento excesivo del almacenamiento dentro del servidor, se implementó un sistema automático de rotación de respaldos. Este procedimiento elimina archivos antiguos conservando únicamente las copias más recientes tanto dentro de la aplicación como en el almacenamiento externo.

Como se observa en la Figura 42, el proceso utiliza tuberías (|) para encadenar múltiples comandos Linux y automatizar la eliminación controlada de directorios antiguos.

Para los respaldos internos de la aplicación, se configuró la conservación exclusiva de las cinco carpetas más recientes:

Para los respaldos internos de la aplicación, se configuró la conservación exclusiva de las cinco carpetas más recientes: **ls -dt */ 2>/dev/null | tail -n +6 | xargs -r rm -rf**

Asimismo, para los archivos comprimidos almacenados externamente, se definió una política de conservación de únicamente diez archivos .tar.gz:

```
ls -t *.tar.gz 2>/dev/null | tail -n +11 | xargs -r rm
```

La combinación de comandos ls, tail, xargs y rm permite identificar automáticamente los archivos más antiguos y eliminarlos sin afectar los respaldos recientes del sistema.

```
# 4.1 Limpieza DENTRO de la app: Mantener solo 5 carpetas
cd "$CARPETA_BACKUPS_APP"
ls -td */ 2>/dev/null | tail -n +6 | xargs -r rm -rf

# 4.2 Limpieza FUERA de la app: Mantener solo 10 archivos .tar.gz
cd "$BACKUP_FUERA"
ls -t *.tar.gz 2>/dev/null | tail -n +11 | xargs -r rm
```

Figura 42. Configuración de rotación automática y limpieza de respaldos antiguos en Linux.

Paso 7. Verificación Manual de Ejecución del Script

Después de finalizar el desarrollo del script backup.sh, se realizó una validación manual del entorno de trabajo para comprobar que la estructura de archivos y directorios necesarios para el sistema de respaldos existiera correctamente dentro del servidor Linux.

Como se observa en la Figura 43, se utilizó el comando ls para listar el contenido del directorio principal del usuario, verificando la presencia del archivo backup.sh, así como de las carpetas backups y unstable, utilizadas durante el proceso automatizado de respaldo.

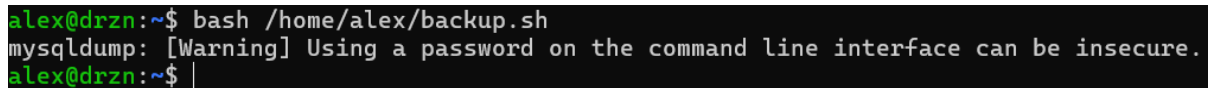
```
alex@drzn:~$ ls
backup.sh  backups  unstable
alex@drzn:~$ |
```

Figura 43. Verificación inicial de archivos y directorios del sistema de respaldos en el servidor Linux.

Posteriormente, se ejecutó manualmente el script de respaldo para validar el funcionamiento completo del sistema automatizado. Tal como se muestra en la Figura 44, se utilizó el siguiente comando: `bash /home/alex/backup.sh`

Durante la ejecución, el sistema realizó automáticamente las tareas de copia de archivos, generación del respaldo de base de datos, compresión y sincronización configuradas previamente dentro del script.

Asimismo, en la Figura 44 se observa una advertencia generada por `mysqldump` relacionada con el uso de contraseñas desde línea de comandos. Esta advertencia corresponde a un comportamiento estándar de MySQL y no afecta la correcta ejecución del proceso de respaldo.

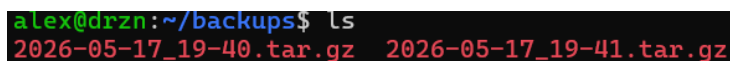


```
alex@drzn:~$ bash /home/alex/backup.sh
mysqldump: [Warning] Using a password on the command line interface can be insecure.
alex@drzn:~$
```

Figura 44. Ejecución manual del script `backup.sh` para validar el funcionamiento del sistema de respaldos.

Después de ejecutar el script, se verificó la creación correcta de archivos comprimidos dentro del directorio `/home/alex/backups`. Como se muestra en la Figura 45, el sistema generó automáticamente archivos `.tar.gz` identificados mediante fecha y hora, permitiendo conservar múltiples versiones históricas de respaldo.

La presencia de estos archivos confirmó que el proceso de compresión y almacenamiento se ejecutó satisfactoriamente.



```
alex@drzn:~/backups$ ls
2026-05-17_19-40.tar.gz  2026-05-17_19-41.tar.gz
```

Figura 45. Verificación de archivos comprimidos generados automáticamente por el sistema de respaldos.

Posteriormente, se validó el contenido interno de las carpetas temporales generadas dentro de la aplicación web. Tal como se observa en la Figura 46, dentro de cada carpeta de respaldo se almacenaron correctamente:

- El directorio `img` con las imágenes de la aplicación.
- El archivo `database.sql` correspondiente al respaldo de la base de datos MySQL.

Esta validación permitió confirmar que tanto los archivos multimedia como la información estructurada de la base de datos fueron incluidos correctamente dentro del proceso automatizado de respaldo.

```
alex@drzn:~/unstable/U3---insecure-webapi/backups$ cd 2026-05-17_19-40
alex@drzn:~/unstable/U3---insecure-webapi/backups/2026-05-17_19-40$ ls
database.sql  img
alex@drzn:~/unstable/U3---insecure-webapi/backups/2026-05-17_19-40$ cd img/
alex@drzn:~/unstable/U3---insecure-webapi/backups/2026-05-17_19-40/img$ ls
1.png  2.jpg
alex@drzn:~/unstable/U3---insecure-webapi/backups/2026-05-17_19-40/img$ |
```

Figura 46. Validación del contenido interno generado por el sistema automatizado de respaldos.

Finalmente, se verificó la sincronización correcta de archivos hacia Google Drive. Como se muestra en la Figura 47, el sistema creó automáticamente la carpeta backups dentro de la unidad de almacenamiento remoto configurada previamente mediante Rclone.



Figura 47. Creación automática de la carpeta backups en Google Drive mediante Rclone.

Asimismo, en la Figura 48 se observa el almacenamiento exitoso de los archivos .tar.gz dentro de dicha carpeta, confirmando que el proceso de sincronización remota se ejecutó correctamente.

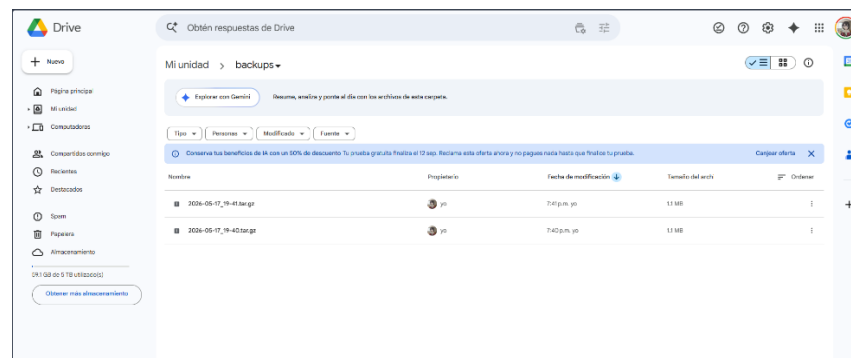


Figura 48. Verificación de sincronización de respaldos comprimidos hacia Google Drive.

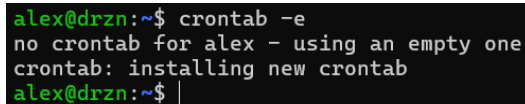
Paso 8. Automatización del Respaldo Mediante Cron

Una vez validado el funcionamiento del sistema de respaldos, se procedió a configurar la ejecución automática del script utilizando cron, herramienta de administración de tareas programadas en sistemas Linux.

El objetivo de esta configuración fue programar la ejecución diaria del respaldo a la 01:01 AM, permitiendo que el proceso se realizara automáticamente sin intervención manual del administrador.

Como se observa en la Figura 49, se utilizó el siguiente comando para acceder al editor de tareas programadas del usuario: `crontab -e`

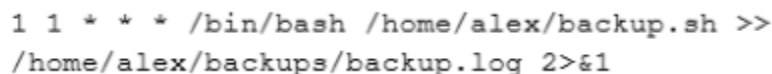
Durante la primera ejecución, el sistema generó automáticamente un nuevo archivo de configuración crontab, el cual posteriormente fue editado utilizando el editor nano.



```
alex@drzn:~$ crontab -e
no crontab for alex - using an empty one
crontab: installing new crontab
alex@drzn:~$ |
```

Figura 49. Acceso y creación inicial del archivo de tareas programadas mediante crontab.

Posteriormente, dentro del archivo de configuración de Cron, se agregó la instrucción encargada de ejecutar automáticamente el script `backup.sh`. Tal como se muestra en la Figura 50, se configuró la siguiente programación:



```
1 1 * * * /bin/bash /home/alex/backup.sh >>
/home/alex/backups/backup.log 2>&1
```

Figura 50. Expresión Cron utilizada para programar la ejecución automática diaria del script de respaldos y almacenamiento de registros del sistema.

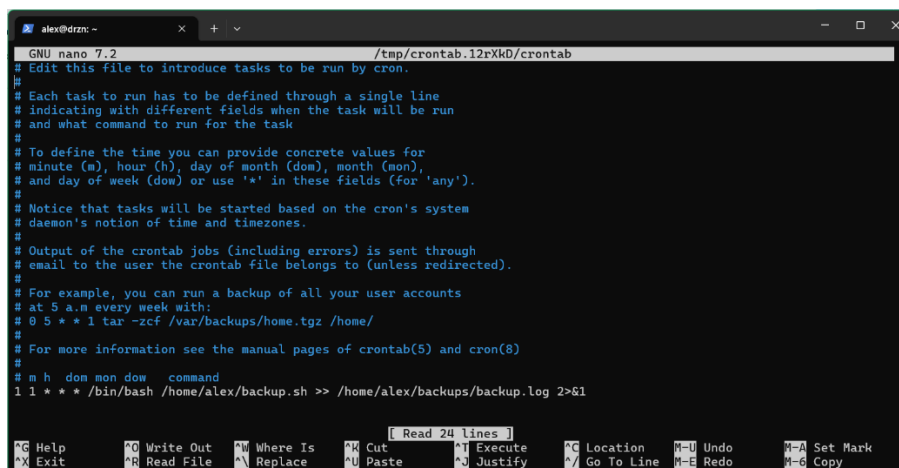
La expresión `1 1 * * *` define el momento exacto de ejecución del proceso automatizado:

- Minuto: 1
- Hora: 1
- Día del mes: * (todos)
- Mes: * (todos)
- Día de la semana: * (todos)

Esta configuración establece que el sistema ejecutará el respaldo diariamente a la 01:01 AM.

Asimismo, dentro de la misma configuración presentada en la Figura 51, se definió la ejecución explícita del intérprete Bash mediante `/bin/bash`, indicando al sistema operativo qué entorno debe utilizar para ejecutar el script automatizado. También se configuró el almacenamiento de registros utilizando el archivo: `/home/alex/backups/backup.log`

La instrucción `>>` permite agregar nueva información al archivo de registro sin sobrescribir el contenido previo, mientras que `2>&1` redirige los mensajes de error hacia el mismo archivo de salida, centralizando todo el historial de ejecución y facilitando futuras tareas de monitoreo y auditoría.



```
alex@drzn: ~  
GNU nano 7.2 /tmp/crontab.12rXk0/crontab  
# Edit this file to introduce tasks to be run by cron.  
#  
# Each task to run has to be defined through a single line  
# indicating with different fields when the task will be run  
# and what command to run for the task  
#  
# To define the time you can provide concrete values for  
# minute (m), hour (h), day of month (dom), month (mon),  
# and day of week (dow) or use '*' in these fields (for 'any').  
#  
# Notice that tasks will be started based on the cron's system  
# daemon's notion of time and timezones.  
#  
# Output of the crontab jobs (including errors) is sent through  
# email to the user the crontab file belongs to (unless redirected).  
#  
# For example, you can run a backup of all your user accounts  
# at 5 a.m every week with:  
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/  
#  
# For more information see the manual pages of crontab(5) and cron(8)  
#  
# m h dom mon dow   command  
1 1 * * * /bin/bash /home/alex/backup.sh >> /home/alex/backups/backup.log 2>&1  
  
[ Read 24 lines ]  
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo       ^_ Set Mark  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  ^E Redo       ^- Copy
```

Figura 51. Configuración de tarea programada para automatizar respaldos diarios mediante Cron.